

Efficient Neural-Symbolic Data System via Multi-Agent Collaboration

Ye Yuan

Beijing Institute of Technology
Beijing, China
yuan-ye@bit.edu.cn

Bo Tang

Southern University of Science and Technology
Shenzhen, China
tangb3@sustech.edu.cn

Zhaojing Luo

Beijing Institute of Technology
Beijing, China
zjliao@bit.edu.cn

Boyang Li

Beijing Institute of Technology
Beijing, China
liboyang@bit.edu.cn

Renjie Liu

Southern University of Science and Technology
Shenzhen, China
liurj2023@mail.sustech.edu.cn

Zhilang Wei

Beijing Institute of Technology
Beijing, China
zhilangwei@bit.edu.cn

Abstract—The paradigm of scaling Large Language and Multimodal Models (LLMs/MLMs) is encountering a significant bottleneck, as the “Scaling Law” of amassing more data and compute becomes unsustainable. To power the next generation of AI, a new paradigm of data systems is required that efficiently fuses the powerful perceptual capabilities of neural networks with the formal, verifiable logic of symbolic systems in order to better manage and analyze data. Recent efforts have demonstrated the potential of this paradigm, yet existing systems remain limited by black-box neural architectures, expensive inference costs, and inadequate support for complex, iterative reasoning workflows over large amounts of multimodal data.

In this work, we envision NSDS, the next-generation Neural-Symbolic Data System, which supports *dynamic, recursive, and multi-stage reasoning* with explainability and lineage auditing over different types of data across heterogeneous programming environments. In particular, NSDS is a hybrid framework combining both agentic and deterministic execution and provides a natural solution for automating task decomposition, iterative refinement, and coordinated execution across neural and symbolic components. We further analyze the key technical challenges in designing such a system, including task decomposition, hardware heterogeneity, multi-agent coordination, unified data storage, and lineage auditing, etc. Finally, we outline the architectural principles that can enable the next-generation neural-symbolic data system.

Index Terms—Neural-Symbolic Data Systems, Large Language Models, Multimodal Analysis, Heterogeneous Execution, AI Data Systems, Lineage Auditing

I. INTRODUCTION

The advent of Large Language Models (LLMs) such as ChatGPT [1] has triggered a revolution in natural language processing, while Multimodal Large Models (MLMs) (e.g., GPT-4o [2]) have demonstrated remarkable capabilities in processing and integrating multimodal data such as text, images, speech, and video. However, this paradigm is encountering a significant bottleneck: both LLMs and emerging large-scale reasoning models demand enormous computational power. Furthermore, LLMs rely on high-quality multimodal data, whereas reasoning models depend on high-quality “thinking data” or symbolic inference. With high-quality data resources

becoming increasingly scarce and computational costs soaring, the “Scaling Law” of simply amassing more data and compute is becoming unsustainable. To power the next breakthrough in AI, a new paradigm of data systems is urgently needed—one that possesses stronger perception and reasoning capabilities while utilizing limited data and computing resources more efficiently. The fusion of neural and symbolic systems, which combines the powerful perceptual capabilities of neural systems with the deep reasoning advantages of symbolic systems, has been shown to be effective in managing and analyzing multimodal data. This neural-symbolic approach is widely explored in academic research [3]–[13] and has already been integrated into major data management systems [14]–[20].

Pioneering models such as DeepSeek-R1 [21] and related research like OpenAI’s “o” series of reasoning models already demonstrate the superiority of the neural-symbolic paradigm. Specifically, the fusion process of neural and symbolic operations relies on both multimodal data (for perceptual training) and detailed “thinking data” (for enhancing reasoning). The reasoning model is a deep-network implementation of the symbolic system. It starts with a complex multimodal reasoning problem and decomposes it into a combination of perception and reasoning tasks. This combination requires fused computation between the two types of complex models, namely multimodal large models (MLMs) and the large reasoning models [22], [23]. However, these implementations for neural-symbolic fusion are based on complex deep neural networks, which are black-box models lacking interpretability [24]–[26] and reproducibility. In addition, the reasoning ability of LLMs corresponds to a finite-variable fragment of first-order logic due to its next-token prediction scheme [27]–[30], and thus cannot reliably express operations such as multi-variable joins, recursive relational operators, and compositional pipelines over heterogeneous data. Thus, LLMs lack the expressiveness required by both structured and unstructured queries. Moreover, they also suffer from context window limits and high costs. For instance, PB-scale industry video datasets typically cannot fit within the context windows of

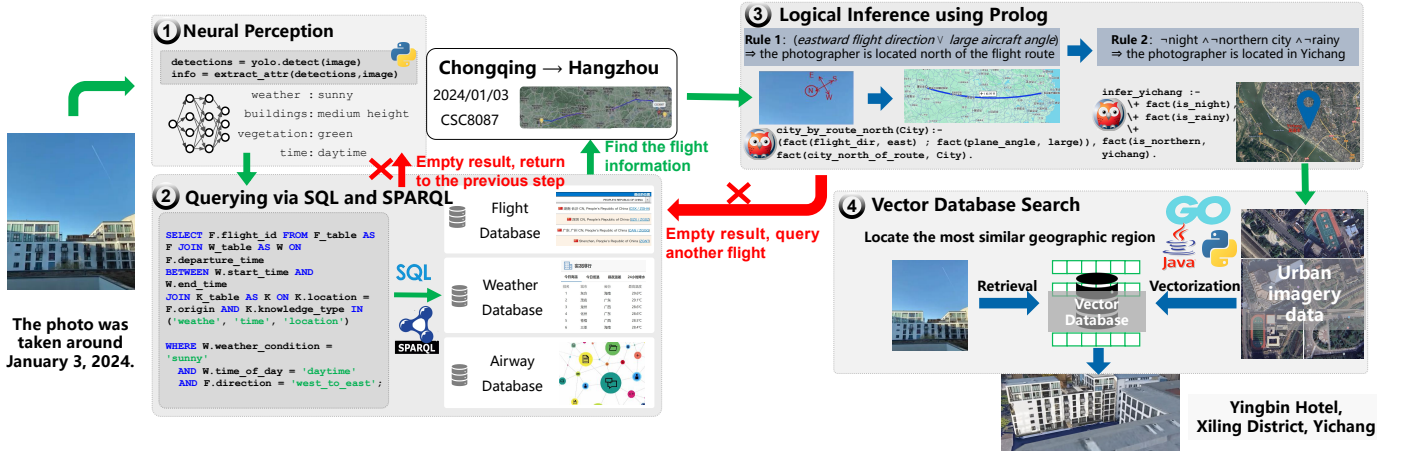


Fig. 1. Example of multimodal intelligence inference

current LLMs and would require tens of hours to process even in batches. Later studies [31]–[36] propose to exploit LLM agents for multimodal data processing and analysis. However, they do not have correctness guarantees as LLMs are observed to have hallucinations and agents lack explicit lineage auditing. Other alternative neural-symbolic integration approaches [37] propose integrating the neural system with the symbolic system in a more lightweight way that either abstracts away the complexities of AI models or exploits simpler symbolic models. While they provide interpretability and incur fewer costs by designing simpler multimodal models and symbolic models, they are designed and tested for simple multimodal and neural-symbolic scenarios which require only single-pass compilation or execution.

Unfortunately, the real-world neural-symbolic applications [9], [38], [39] are much more complex. For example, the real neural-symbolic analysis typically involves a sequence of interacting sub-tasks among different multimodal data systems (e.g., graph databases, relational databases, vector databases, image databases, etc.) across *heterogeneous programming environments*, and the analysis process is inherently *dynamic*, *recursive*, and *iterative*. Additionally, since the initially generated execution plan is unlikely to be optimal or fully correct, the plan must be refined and adjusted through multiple stages of reasoning. This involves repeated cycles of *plan generation*, *performance optimization*, *error revision*, *backtracking*, and *iterative execution* among different data systems. However, none of the existing neural-symbolic data analytics systems consider these complex scenarios.

In this vision paper, we first showcase representative neural-symbolic applications, including complex multimodal intelligence inference (e.g., identifying the corresponding flight from a landscape photo by fusing visual cues with flight databases) and neural-symbolic video analysis over massive video clips (e.g., finding all clips that reveal desired information). We then summarize the key challenges in these applications, which provide insights that guide the design of our proposed NSDS, the next-generation Neural-Symbolic

Data System. We next explore the vision and architecture of NSDS. Specifically, to address the need to tackle *heterogeneous programming environments* and *the complex iterative analysis process* across different types of data systems in neural-symbolic applications, we believe that a hybrid framework integrating agentic exploration and deterministic execution, in which agents are endowed with autonomous planning, adaptive exploration, and coordination capabilities, provides a natural and effective paradigm for automating the exploration, backtracking, and refinement cycles inherent in neural-symbolic task analysis. By combining advanced neural reasoning and efficient symbolic processing, this hybrid system can perform advanced reasoning beyond the finite-variable fragment of first-order logic, and can realize complex recursive operations. Moreover, NSDS supports both structured and unstructured inference over unlimited-length context with efficient and deterministic operators. Additionally, the combination of deterministic operators provides necessary transparency for complex neural-symbolic executions, enables lineage auditing, and improves end-to-end system efficiency. However, designing such a new neural-symbolic data system with a hybrid framework still faces many technical challenges, including effective task decomposition within heterogeneous programming environments, multi-objective optimization of neural and symbolic analysis, efficient dispatching of heterogeneous hardware, and complex coordination among different agents. In this work, we provide a comprehensive discussion of these core technical challenges and propose novel architectural principles to address them, laying the groundwork for the next generation of neural-symbolic data systems. Finally, we demonstrate the efficiency of the NSDS prototype on real-world reasoning tasks, such as inferring the precise location shown in a real-world image and extracting key information from a vast number of video clips.

II. USE CASES

In this section, we introduce two representative neural-symbolic applications (i.e., multimodal intelligence inference

and neural-symbolic video analysis) in the real world.

A. Multimodal Intelligence Inference

Deriving complex intelligence information from multimodal data is an extremely valuable yet challenging task. Such tasks typically require a deep integration of the visual information and structured factual data [40]–[42]. Traditional multimodal inference approaches rely heavily on manual operations performed by domain experts [43], [44]. Experts must first deconstruct the image using their professional knowledge, extract and interpret the visual information, and then manually search for and cross-verify data in specialized databases. This process is not only time-consuming and labor-intensive but also highly dependent on individual expertise and knowledge base. Any subjective misjudgment may lead to erroneous queries or inaccurate results. A neural-symbolic system fundamentally restructures this workflow, realizing a multilingual, end-to-end, automated, and verifiable reasoning pipeline. For instance, given an image and its shooting time, the task is to infer the photographer's location. Figure 1 depicts the overall inference process, and we elaborate on the key steps as follows.

Step 1: Neural Perception. The system first employs Python-based neural networks to identify visual elements within the image and determine their specific attributes and priorities (e.g., weather conditions, solar elevation angle, flight direction, vegetation density, etc.). In this example, the system determines that the weather is sunny, vegetation is green, the time is daytime, the buildings are of medium height, and the white streak represents a flight trajectory.

Step 2: Querying via SQL and SPARQL. Next, the system uses SQL to query the relational database storing flight information and SPARQL to query the graph database storing flight route information in order to determine the corresponding flight. It first employs a graph database to identify potential flight routes and then, based on the flight path, weather, and temporal factors, retrieves candidate flight data from the relational database. The system then determines that the flight is CSC8087, traveling from Chongqing to Hangzhou, China on January 3, 2024.

Step 3: Logical Inference using Prolog. Based on the confirmed flight information, the system performs rule-based reasoning using Prolog. The reasoning logic follows: $(\text{eastward flight direction} \vee \text{large aircraft angle}) \Rightarrow$ the photographer is located north of the flight route; and $\neg \text{night} \wedge \neg \text{northern city} \wedge \neg \text{rainy} \Rightarrow$ the photographer is located in Yichang, China. To be specific, if the flight is heading east or the aircraft's viewing angle is relatively large, it is inferred that the photographer must be positioned to the north of the flight path. Furthermore, given that it is neither nighttime nor raining, and that the city is not situated in a northern region, the system deduces that the most probable shooting location corresponds to Yichang, China.

Step 4: Vector Database Search. Finally, the system performs a similarity-based retrieval in a vector database to

determine the photographer's location. The visual and contextual features inferred in previous steps are encoded into a dense query vector, which is compared against pre-indexed geographic embeddings representing landmarks and environmental conditions in Yichang, China. By computing similarity scores, the system retrieves and ranks candidate regions that most closely match the inferred scene characteristics, and ultimately identifies the final location as Yingbin Hotel, Xiling District, Yichang, China.

Complex Multi-Round Execution and Backtracking. As shown by the red arrows in Figure 1, the entire inference process is not strictly linear but dynamically adaptive. When an execution path encounters an error, incomplete data, or failed reasoning, NSDS automatically performs backtracking to previous steps and re-evaluates alternative reasoning directions. Taking Steps 2 and 3 as an illustrative example, if the query returns no results in Step 2, the inference path may be incorrect, and NSDS will return to the previous step to adjust its reasoning direction. In Step 3, if multiple candidate flights exist, the current inference result may be undefined, prompting NSDS to return to Step 2 to examine other valid flight candidates. This requires the system to conduct complex multi-round executions in which each iteration leverages feedback from prior attempts to refine hypotheses, update query parameters, or select new inference paths.

Compared with existing LLM-based approaches (e.g., GPT-5.2) or domain expert-based approaches, NSDS demonstrates significant advantages in multimodal reasoning tasks. It first completely eliminates dependence on manual analysis by domain experts, automatically optimizes operator selection and updates, and enables multilingual hybrid reasoning. As shown in Section V, the latency and explainability of the above reasoning tasks of NSDS are obviously better than GPT-5.2, especially with over 70x speedup in system efficiency.

B. Neural-Symbolic Video Analytics System

Video is becoming one of the predominant sources of unstructured information in modern enterprises. Video analysis is highly challenging as it involves massive data volumes and the semantic information is deeply embedded within pixel-level details and temporal variations. As a representative video analysis task, consider finding clips from the past decade of the three major U.S. cable news video archives in which Jake Tapper interviews an angry Bernie Sanders.

Mainstream video analysis solutions mostly rely on the reasoning capability of LLMs, with the models being invoked on every clip in the entire video dataset, leading to prohibitive inference costs. More importantly, industry-scale video workloads, which contain billions of video clips, typically exceed the context window limits of current LLMs, causing severe scaling issues in real applications.

In contrast, NSDS offers a unified implementation scheme. It treats neural network models as native operators directly invoked by the query language. By allowing models, such as object detection and emotion recognition, to be embedded within SQL-like first-order-logic queries, it enables the formal

expression of complex video semantic retrieval. Furthermore, NSDS achieves a unified execution plan for both SQL and neural network operators. By combining CPU-GPU cooperative scheduling with cost-based optimization strategies, it jointly optimizes for both accuracy and performance. At the storage layer, NSDS enables efficient search indexing over billions of video clips and efficient management of video metadata—capabilities not supported by current LLMs—for scalable, robust, and low-latency video retrieval and analytics.

In summary, NSDS deeply fuses the strengths of traditional databases in structured data management with the capabilities of deep learning models in unstructured data understanding, providing a system-level, unified analysis framework for complex data types like video. It not only makes complex video analysis feasible on industry-scale datasets but also enables further system efficiency speedups through unified optimization. By this means, it delivers a more scalable and reliable infrastructure for large-scale intelligent video analysis. Experimental results in Section V further confirm the advantage of NSDS over GPT-5.2, with a speedup of about 90x in end-to-end latency.

III. INSIGHTS FROM NEURAL-SYMBOLIC USE CASES

The aforementioned use cases show distinct characteristics and features for neural-symbolic data analytics tasks. In this section, we discuss the insights derived from these use cases, which are valuable for guiding the design of our NSDS. Figure 2 abstracts the main process of the end-to-end neural-symbolic data analytics. Specifically, neural-symbolic data analytics involves decomposing complex reasoning problems into a series of executable sub-tasks that integrate both neural perception and symbolic reasoning, and formulating an optimal execution plan across perception and reasoning stages. The main objective is to combine the strengths of neural networks in perception, representation learning, and generalization with the advantages of symbolic systems in logical reasoning and interpretability, thereby achieving efficient and interpretable end-to-end intelligence. To develop a robust, efficient, and reliable NSDS for the aforementioned applications, several challenges remain.

Heterogeneous Programming Environments. The neural-symbolic data analytics process follows a paradigm consistent with that of large language reasoning models, where users interact with the system through natural language, and the system subsequently compiles these instructions into a sequence of sub-tasks in **heterogeneous programming or logical forms**, such as Python, SQL, and Prolog. Essentially, it is a translation and planning process that spans intent understanding, semantic parsing, and program synthesis across **heterogeneous compilers**.

Complex and Iterative Execution. Unlike traditional single-pass compilation or execution [45], [46], neural-symbolic analytics tasks are inherently **dynamic, recursive, and iterative**. Since the initial plan generation is unlikely to be optimal or fully correct, the system must refine and adjust the plan

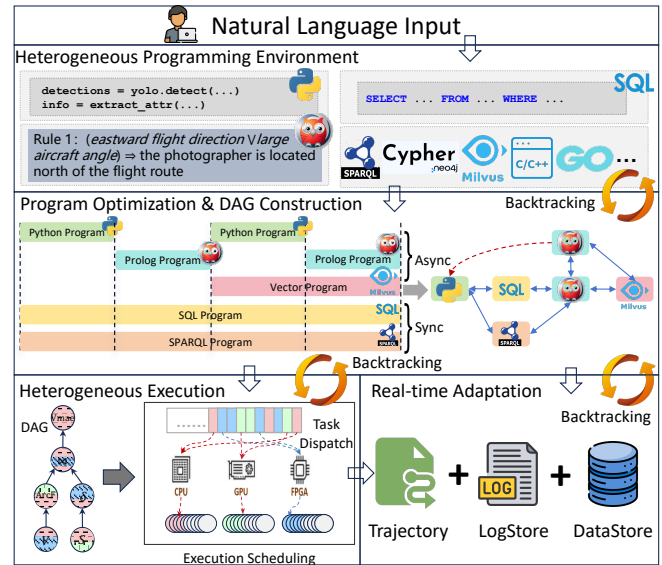


Fig. 2. Characteristics of end-to-end Neural-Symbolic processing tasks.

through multiple stages of reasoning. This involves repeated cycles of **plan generation, performance optimization, error revision, backtracking, and iterative execution**. Through such multi-stage refinements, the system converges toward an execution plan integrating neural and symbolic operations that satisfy task constraints while maintaining efficiency and interpretability, achieving adaptive reasoning and executable intelligence from natural language to actionable behavior.

Heterogeneous Hardware Dispatching. Since neural and symbolic tasks are executed on fundamentally different types of hardware, NSDS must also ensure efficient, coordinated computation across heterogeneous environments. In particular, multimodal perception and representation learning typically rely on GPU-accelerated deep neural networks, which are optimized for large-scale parallel tensor operations [47], [48]. In contrast, symbolic reasoning, logical planning, and rule-based inference are generally executed on CPUs, where control-flow-intensive and branching-heavy computations are more efficient. This heterogeneity introduces non-trivial challenges, including data movement between GPU and CPU memory spaces, synchronization overheads, and the need for dynamic scheduling strategies that allocate workloads to the most suitable hardware resources.

Real-time Adaptation and Optimization. Since neural reasoning execution is internally unstable and thus needs interleaved revisions and backtracking, agent decisions should be continuously optimized by leveraging historical interaction trajectories [49]. Specifically, by logging and checkpointing historical interactions, including execution failures, unexpected results, and successful workflows, those experiences should be utilized in real time as helpful training data for the neural components to support continuous learning in both decision-making and action selection. Moreover, recorded workflows

are also valuable references for status checking and lineage auditing, providing system transparency and explainability. In addition, the underlying data storage should also be adjusted and reorganized for better data access locality based on real-time access distribution.

IV. CHALLENGES AND OPPORTUNITIES

In this section, we first provide an overview of our NSDS, and then discuss the challenges and opportunities involved in building such an efficient neural-symbolic data system.

A. System Overview

Building upon this complex and iterative process, we argue that a hybrid framework that combines both agentic exploration and deterministic execution provides a natural and effective paradigm for the requirements inherent in neural-symbolic analytics tasks. This hybrid framework boasts advantages including (1) **adaptively determining the number of agents and their roles based on the complexity of the task**; (2) **efficiently executing the constructed workflows and ensuring answer correctness**; (3) **automating the exploration, backtracking, and refinement cycles among agents**. This makes it a natural fit for the neural-symbolic workload, which is inherently dynamic, iterative, and heterogeneous. Instead of a rigid, linear script, a hybrid framework uses a combination of specialized agents and deterministic workflows to autonomously manage complex, multi-language task execution while ensuring efficient, correct execution. Specifically, each agent is endowed with autonomous planning, adaptive exploration, and coordination capabilities, enabling the system to dynamically decompose, assign, and optimize sub-tasks. Meanwhile, for each sub-task, subsequent deterministic optimization and execution ensure efficient and robust execution in heterogeneous environments.

Figure 3 shows the proposed hybrid framework of NSDS, depicting two distinct types of agents and three deterministic stages. In the agentic part, this hybrid framework consists of the *decomposition* and the *coordination* agents. In particular, the *decomposition* agent decomposes tasks into unified operators for different applications. Meanwhile, the *coordination* agent ensures strong global consistency, facilitating better control over user-specific constraints and SLOs (such as latency, cost, and accuracy) with efficient inspection and backtracking. Regarding the deterministic stages, optimization and execution enable us to leverage well-studied principles to achieve optimal efficiency under heterogeneous workloads while ensuring answer correctness. Additionally, unified backend analysis provides insight into the system's runtime status through performance profiling and end-to-end workflow inspection.

Given a new user query, the *decomposition agent* first automatically explores possible combinations from the set of unified operators (UOPs) of heterogeneous programming languages, e.g., Python, C++, Prolog, SQL, etc., and then generates a series of unique, clearly defined sub-tasks that can accomplish the user's task. The *optimization stage*

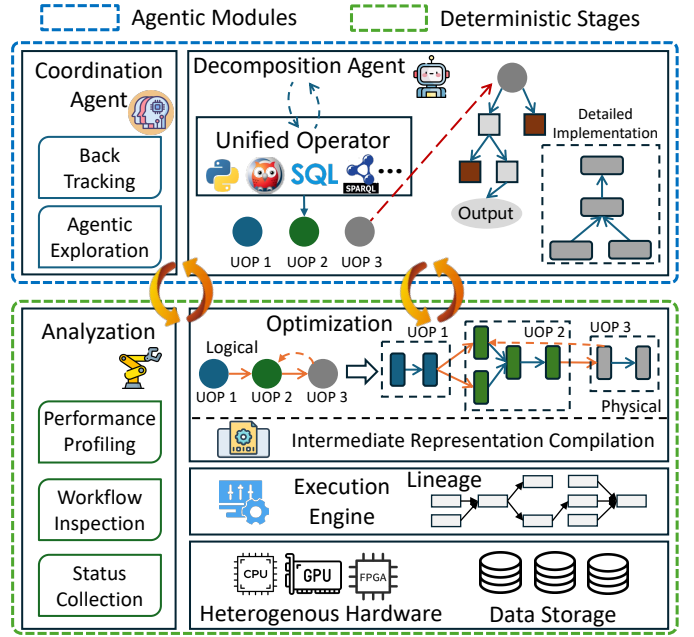


Fig. 3. Overview of our agentic-deterministic hybrid NSDS framework.

inspects the overall pipeline between programs and constructs a data flow graph (DAG) that captures the program calling sequence. Moreover, it also automatically plans and proposes multi-pass optimizations on the DAG, including both logical and physical optimizations with learning capabilities such as learned cardinality estimation and learned indexes, and finally performs IR compilation to unify the execution of different programs. After an optimized DAG is yielded, multiple *executions* are invoked for obtaining program results, which adaptively dispatch programs of each domain to the underlying heterogeneous hardware and schedule the execution for better efficiency with real-time feedback. Due to the complex nature of the neural-symbolic analytics process, we cannot guarantee that the whole process is executed correctly at one time. As a result, the iterative backtracking and refinement among different agents are necessary. We thus design the *coordination agent* to collectively receive feedback signals, determine the point of failure, and invoke rollbacks or retries to ensure end-to-end correctness through multi-round tentative attempts. Alongside the entire workflow, the *analysis stage* inspects the entire execution profile and logs it to the database storage layer, including the complete DAG, program latency, resource occupation, and result size, which facilitates lineage auditing.

B. Cross-domain Task Decomposition

The first major challenge to be solved in NSDS is how to provide users with a low-barrier, user-friendly approach to translate a single, complex query intent (e.g., expressed in natural language) into an executable plan. This plan is not a single program but a sequence of sub-tasks that must be executed across **heterogeneous programming and logical forms**,

such as `Python` for neural perception, `SQL` for relational data querying, and `Prolog` for logical rule-based inference. This polyglot requirement poses a significant barrier, as end-users cannot be expected to be experts in AI, database systems, and symbolic logic. To lower this barrier, the system must translate the user's intent into these disparate environments, which necessitates the following foundational components:

(1) *A Unified Operator (UOP) Abstraction*: The system needs an abstraction and registration mechanism where operators from all environments can be defined and integrated. This allows users to register custom computational operators—whether it is a `Python`-based neural model, a `Java` function, or a pre-defined `SQL` query template—and treat them as building blocks. Some representative UOPs include neural operators, LLM operators, symbolic operators (e.g., logical operators, `SQL` operators), and so on.

(2) *A Hybrid Query Grammar*: The system requires a neural-symbolic native set of lexical and grammatical rules. This grammar is crucial for supporting complex queries that involve the nested integration of these heterogeneous operators, such as taking the output of a `Python` perception module and correctly feeding it as a parameter to a `SQL WHERE` clause.

(3) *A Polyglot Semantic Analyzer*: A unified semantic analyzer is needed to validate the *entire* hybrid plan. This analyzer must understand the semantics of all involved environments to verify the correctness and completeness of the abstract syntax representation, ensuring, for example, that the data passed between `Prolog` and `SQL` is compatible.

The second and more fundamental challenge lies in establishing a formally sound and expressively complete algebraic system that **unifies the mutually alien algebras** of these different programming paradigms. The foundational paradigms are fundamentally disparate. Neural computation, typically implemented in `Python`, is implicit and continuous. In contrast, the symbolic logic relied upon by database systems, expressed in `SQL` or `Prolog`, is inherently explicit and discrete. To ensure reliable execution, a novel **fused algebraic system** must be formalized. This algebra acts as a universal logic representation of operators from different domains. It should precisely define neural operators (like `Python` functions) and symbolic operators (like the relational algebra behind `SQL`) and establish the hybrid operational rules governing their interaction. This formal system should then be validated for completeness to ensure that as we support complex, cross-modal reasoning, we preserve the established database advantages of interpretability and determinism.

C. Data Flow Construction And IR Optimization

Efficient execution of heterogeneous programs represented by UOPs in NSDS presents a key challenge in the design of a hybrid neural-symbolic data processing framework. The key is to construct the data flow among UOPs and *jointly optimize this end-to-end pipeline*. Efficient execution depends not just on the individual UOP but on how this data flow is physically ordered and executed as a single, unified workflow.

This task renders traditional optimizers insufficient, as they

are typically designed to optimize a *single* language against a *single* objective (e.g., efficiency) [50]. Native optimization for NSDS, in contrast, must jointly optimize the entire data flow of heterogeneous programs against *multiple complementary goals*, including overall pipeline efficiency, the accuracy contributed by neural operators, and the execution cost (e.g., GPU vs. CPU time or cloud service fees). Moreover, a customized Intermediate Representation (IR) for UOPs should also be developed to compile disparate front-end operators into a shared low-level representation that can be optimized and directly guide subsequent execution scheduling. As such, this joint, multi-objective optimization necessitates not just logical rules based on syntax parsing and analysis, but also physical practices adaptive to the underlying data and hardware. To achieve these complex optimization goals, four core challenges must be addressed.

The primary task in this optimization stage is to establish a comprehensive and dynamic set of optimization rules that define the transformation space for the entire multi-language data flow. This requires supporting modularized optimization rules that operate both *within* and *across* program boundaries. Examples include traditional query rewriting, neural operator fusion, and, critically, equivalent transformations across language boundaries (e.g., formally proving that a `Python`-based filter can be rewritten as an equivalent `SQL WHERE` clause to be “pushed” into the database). To handle the uncertainty of neural operators, the commutative and distributive laws governing these cross-language transformations must be formalized to guarantee semantic equivalence.

Second, this optimization stage requires a data- and model-aware cost estimation model that can *jointly* price the entire heterogeneous pipeline. This model must deeply integrate the determinism of symbolic systems with the dynamic behavior of neural systems. The research lies in building a multi-dimensional cost model with learning abilities that, for example, can predict how changing a model's parameters (affecting its accuracy and runtime) will propagate and influence the cost and data size processed by a subsequent table join. This cost model should consider the hybrid storage layout across the underlying multimodal data systems (e.g., graph databases, relational databases, vector databases, image databases, etc.), and provide insights for adaptive selection of unified or learned indexes. This model must also be hardware-aware, accurately capturing the runtime trade-offs of executing a neural model task on a GPU versus a symbolic task on a CPU.

Third, this optimization stage should address multi-objective decision-making for the *entire pipeline*. This stage needs to first design efficient algorithms to approximate the Pareto-optimal frontier, identifying a set of pipeline configurations that offer the best trade-offs between conflicting goals (e.g., high-accuracy/high-cost neural models vs. low-accuracy/low-cost models). Second, it should construct a decision model to select the *final* end-to-end execution plan from this frontier, ensuring an optimal balance between total query performance, result accuracy, and resource cost for the complete, multi-language workflow.

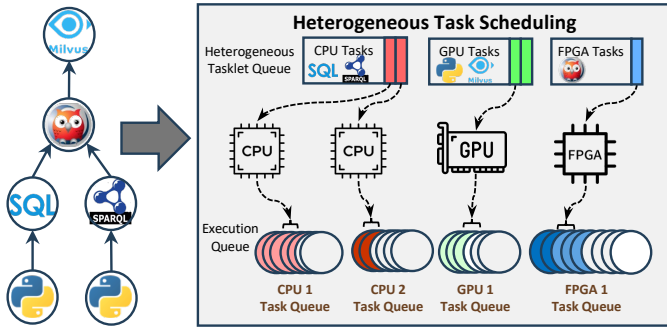


Fig. 4. DAG execution scheduling of cross-domain programs on heterogeneous hardware environments.

Finally, this optimization stage should compile the workflow of unified operators in heterogeneous programming languages to a shared low-level intermediate representation (IR) that can apply general physical optimization and directly guide the subsequent execution and scheduling of hardware. Rather than relying on fragile, manually written glue codes to connect these distinct domains, the IR for neural-symbolic tasks should natively capture the core data dependencies, control flows, and operator relations across the entire neural-symbolic analytics pipeline in a unified schema. Specifically, existing intermediate representations are typically designed for specific programming languages. In contrast, our IR targets neural-symbolic operators and explicitly captures the semantics of neural-symbolic computation, including conflicting optimization objectives and heterogeneous programming paradigms. By enabling cross-language and cross-semantic IR learning, this design lays the foundation for efficient and unified multilingual neural-symbolic collaboration and execution.

D. Heterogeneous Execution

NSDS needs to ensure its efficient physical execution on the underlying heterogeneous hardware environments. As shown in Figure 4, this involves jointly **dispatching** and **scheduling** the optimized programs in the pipeline onto a diverse set of hardware, including CPUs, GPUs, and FPGAs. During this process, a dynamic execution scheduler is needed to account for the large scheduling space of the whole set of programs. As such, multiple execution workers need to collaboratively consider the optimal execution order and resource allocation, and adaptively adjust the policy with real-time performance feedback. In these complex execution environments, several concrete challenges arise.

The first challenge is the inefficiency of data movement in traditional Von Neumann architectures. In our neural-symbolic pipeline, workloads frequently alternate between different hardware types, for example, executing a neural computation on a GPU, followed by a symbolic reasoning task on a CPU. These frequent data movements between devices (e.g., shuffling data from GPU memory to CPU memory) incur substantial communication latency and energy overhead, emerging as the primary bottleneck constraining

overall system performance. To solve this, a promising direction is to move towards a **data-centric and near-data execution paradigm**. By jointly modeling the data transfer and the computation costs for each program, the system can dynamically **schedule** and optimally **dispatch** workloads. For instance, the scheduler must decide if the cost of moving data to a GPU for a model inference task is worthwhile, or if it is cheaper to execute a (slower) CPU-based implementation of that task. This requires abstracting all capable devices into a unified execution layer, which can then allocate each program in the DAG to the most suitable execution unit.

Second, the diversity of hardware introduces immense complexity, as each type of hardware (e.g., CPU, GPU, FPGA) has different programming models, memory hierarchies, and data format requirements. A neural operator optimized for a GPU (e.g., using CUDA) has a completely different implementation than one for a CPU. To manage this, a promising approach is to design **hardware-affine minimal execution units** and portable, high-performance operator libraries. This allows the system to map a single UOP in the DAG to multiple physical implementations, enabling the scheduler to dynamically choose the best one. This is governed by a **heterogeneous execution DAG model** that accurately represents both task dependencies and hardware affinity.

Third, existing load balancing schedulers are often inadequate, as they fail to consider both the characteristics of the *program* and the *hardware* (e.g., CPU and GPU memory bandwidth). An efficient scheduler must perform load balancing at two levels: 1) It must intelligently dispatch computational tasks based on hardware-specific characteristics. For example, it must balance the stream of model inference tasks to the GPU against the logical reasoning tasks being sent to the multi-core CPU; 2) It must also achieve load balancing *within* a single component, such as using threads to parallelize logical reasoning tasks across multiple CPU cores. The above two requirements advocate for a resource scheduler that performs dynamic load monitoring (e.g., collecting real-time CPU utilization and GPU memory usage) to promptly detect imbalances and automatically adjust task allocation.

E. Multi-agent Coordination

This NSDS framework is characterized by its composition of multiple, specialized, autonomous agents. This distributed, agentic execution is inherently unstable and complex, yet it must fulfill the exploration, backtracking, and refinement cycles essential for neural-symbolic tasks. The primary *need*, therefore, is to ensure end-to-end correctness across this entire collaborative pipeline. This creates a significant *challenge*: a failure at any single point can jeopardize the entire workflow. A robust coordination mechanism is thus required to manage this dynamic and error-prone process.

The first challenge is *failure detection and localization*. In a distributed system of agents, simply knowing the existence of an error is insufficient; the system must determine *which* agent is responsible and *why the failure occurred*. This is *highly challenging*, as the fault may not originate with the

failing agent but could be the result of error propagation from another agent's decision, making true root cause analysis extremely difficult [49], [51], [52]. A promising solution is the introduction of a dedicated *coordination agent*. This agent acts as a central nervous system for the framework, designed to collectively receive feedback signals from all other participating agents. This centralized feedback mechanism is what allows the coordination agent to effectively determine the point of failure. For example, if the optimization stage cannot generate a reasonable DAG, feedback is raised and captured by the coordination agent for real-time and multi-turn failure identification and recovery.

The second challenge is *stateful recovery and automated backtracking*. Once a failure is localized, the system must recover without discarding the entire computational effort. The coordinator agent addresses this by being empowered to *invoke intelligent rollbacks or retries*. This function is the key to automating the exploration, backtracking and refinement cycles. Instead of a catastrophic failure, the coordination agent initiates targeted recovery actions based on the feedback.

F. Unified Data Storage

The unified operators in our NSDS span across heterogeneous programming environments. As a result, they are executed on diverse data sources consisting of multiple data modalities. To support all unified operators and enable holistic optimization in a single framework, NSDS first needs to extract relevant data sources, such as geolocation data, from remote sources (e.g., third parties) into unified local storage in an offline manner. As such, the storage layer of NSDS also faces heterogeneous challenges of different data sources. Current multimodal data storage systems are often “data silos”, storing different modalities in isolated formats. Unstructured data, in particular, often lacks the metadata needed for discovery, leading to a “data swamp”. To overcome this, a new multimodal storage abstraction should have an integrated data storage design, including considerations for a unified storage format and efficient cross-modal indexes. To achieve this design, three primary challenges arise: 1) establishing a *joint representation* for cross-modal data; 2) designing an *efficient and integrated layout*; and 3) ensuring the storage format is *extensible* for future workloads.

The first challenge is enabling *rapid and unified cross-modal retrieval*. Neural-symbolic processing requires simultaneous access to diverse data types, such as images for model inference and structured tables for symbolic reasoning. Current systems are inefficient, using isolated methods and a patchwork of disparate indices (e.g., B+ trees, vector indices) that are difficult to manage and costly to query. A promising solution is to develop a *joint representation model* that uniformly encodes structured, semi-structured, and unstructured data blocks into the same high-dimensional vector space. This enables unified discovery and indexing and can be coupled with a *hybrid indexing strategy* and an optimizer that selects the most efficient access order to reduce I/O.

The second challenge is designing an *efficient and integrated data layout* that supports the mixed access patterns of neural-symbolic queries. These queries demand a mix of fast scanning, fine-grained random access, and efficient I/O for large unstructured data. Current solutions are inadequate: vector databases scatter raw data, while standard columnar formats (like Parquet or ORC) lack optimization for unstructured data, have mismatched access patterns, and limited parallel processing capabilities. The proposed solution is to explore a *unified, integrated storage format* that targets new-generation hardware and supports techniques like GPU-friendly encoding/decoding, sparse vector quantization, and lightweight compression.

The third challenge is ensuring the *extensibility and interoperability* of the storage format. Data originates from many applications, and the query engine must be able to read it without knowing the custom, domain-specific encoding methods used to store it. This is “impractical” with current file readers, which lack a unified API and must be manually updated for every new encoding method. The solution is to create a *standardized and extensible data decoding API*. A promising approach is to allow the storage file to contain its decoding logic in a self-contained manner, for example, by *compiling data decoding into space-efficient bytecode* and embedding it within the file itself.

G. Continuous Learning and Real-time Adaptation

To adapt to unseen environments in production settings, this agentic-deterministic hybrid system demands a robust mechanism for *continuous learning and optimization*. This need extends to every agent and stage, targeting improvements in both the *correctness* of their individual outputs and the *efficiency* of the entire pipeline to meet user constraints like Service-level Objectives (SLOs). This requirement necessitates the introduction of the *analyzation* stage. As shown in Figure 3, this stage is responsible for status collection, performance profiling, and lineage auditing. Its core function is to act as the system’s memory by logging and checkpointing the execution status of the pipeline and storing the agent interaction trajectories. This data can be further used to tune the coordination and decomposition agents above for continuous learning and provide interpretability by reviewing the workflow lineage.

To fulfill such requirements, the first challenge is ensuring *action correctness*, as a single flawed output, such as an incomplete task set from the decomposition agent, can cause the entire pipeline to fail. A promising solution to address this is to log and checkpoint execution failures, unexpected results, and successful workflows. This creates an invaluable historical dataset, which can be used as a post-training corpus or as in-context examples. This allows the system to leverage in-context learning or reinforcement learning techniques, enabling agents to learn from exploration and past failures, thereby improving their ability to produce the desired output in subsequent tasks.

The second challenge is achieving *holistic system robustness*. It requires the system to produce valid output that can be recognized and processed for any given user input. A promising solution here for the analyzation stage is to summarize the vulnerabilities of the current execution pipeline by analyzing the complete DAG and interaction trajectories from a large corpus of runs. This aggregated analysis provides feedback for holistic optimization from the system perspective. For example, this data can be used to retrain the decomposition agent to avoid brittle decomposition strategies or to help the coordination agent learn more effective rollback policies.

The third challenge is *efficiency and real-time adaptation* to meet performance goals. A logically correct and robust pipeline may still lack efficiency, violating user SLOs. As such, the analyzation stage should be able to tackle this by logging quantitative performance metrics, including the complete execution DAG, program latency, resource occupation, data access distribution, etc. This enables a two-pronged optimization solution. First, this execution profile could be fed back to the optimization stage, allowing it to learn to generate more efficient DAGs. Second, the access patterns could be used to trigger real-time adaptation, allowing the system to dynamically adjust the data layout of the database storage layer for better access locality.

H. Lineage Auditing

Beyond performance monitoring, a critical requirement for the system’s reliability is the implementation of comprehensive *lineage auditing* across the entire hybrid pipeline. Because the interaction between agents is inherently unstable and the neural components often operate as black boxes, the path from a natural language query to the final result involves a complex series of stochastic transitions. Lineage auditing must therefore capture the *complete provenance of every decision*, including tracking the specific logic used by the decomposition agent, the subsequent physical execution paths taken on heterogeneous hardware, the concrete parameters passed to each function call, and every intermediate result generated throughout the execution process. This granular lineage auditing provides the necessary interpretability for dynamic agentic execution and possible root cause analysis when failures occur, allowing both manual inspection and self-refinement by pinpointing where and why failures occur. Ultimately, by transforming these historical trajectories into verifiable reasoning chains, lineage auditing serves as a high-quality feedback loop that enables the system to refine its logical consistency and maintain interpretability and accountability in real-world deployments.

For example, in the multimodal inference workload illustrated in Figure 1, the lineage auditing module in NSDS will need to capture the complete provenance of the reasoning chain to ensure transparency and reproducibility. Specifically, it would record the initial Neural Perception metadata, such as the specific version of the Python-based neural network model and the confidence scores for identified visual elements like “green vegetation” or “daytime”. It would then track the

TABLE I
EXPERIMENTAL RESULTS ON THE GEOLOCATION INFERENCE TASK

Time Breakdown	NSDS	GPT-5.2
Image Detection	0.125s	—
SQL Generation	3.420s	—
SQL Execution	0.115s	—
Result Aggregation	0.010s	—
End-to-end Time	3.670s	209.0s

TABLE II
EXPERIMENTAL RESULTS ON THE VIDEO QUERYING TASK

Time Breakdown	NSDS	GPT-5.2
SQL Generation	22.93s	—
SQL Execution	10.84s	—
Frame Detection	45.13s	—
End-to-end Time	78.9s	7047.80s

transition to symbolic querying, logging the exact SQL and SPARQL strings generated to identify flight CSC8087, the version of the relational and graph databases accessed, and the intermediate results returned. Furthermore, the lineage would include the discrete Prolog rules applied during logical inference such as the specific aircraft angle parameters, and would also record the resulting geometric deduction that placed the photographer north of the flight route. Most critically, the lineage auditing would document any backtracking events, capturing the “empty results” that triggered a return to previous steps or the re-evaluation of alternative flight candidates. By recording the concrete parameters across heterogeneous environments and the intermediate results generated, the system provides an interpretable and verifiable reasoning chain that allows for precise workflow inspection.

V. EXPERIMENTAL EVALUATION

To assess the neural-symbolic analysis capability of NSDS, we conducted two experiments. The first is a geolocation inference task, which aims to infer the precise location of a real-world image containing multiple semantic entities. The second is a video querying task, which targets retrieving video segments that satisfy a set of complex conditions from a large-scale video corpus. The baseline is GPT-5.2, a state-of-the-art large language model.

A. Geolocation Inference Task

For the geolocation inference task, given a new image, NSDS first generates the execution plan automatically via the collaboration of decomposition agents and the optimization stage. The execution engine subsequently performs iterative execution, which involves object detection in Python, automatic SQL generation via an LLM based on detected objects, and SQL execution over an RDBMS storing the geolocation information. Given the complex interactions between object detection and SQL generation, errors such as empty result sets or incorrectly generated queries occur inevitably. This

execution process thus involves multi-round execution and backtracking, which is automatically handled by our coordination agent. Finally, the query results are aggregated and ranked to derive the inference outcome, thereby completing the symbolic reasoning process. In comparison, the baseline uses GPT-5.2 alone to perform end-to-end visual-linguistic reasoning. The comparison in terms of running time is shown in Table I. From the table, we observe that it takes NSDS only 3.67 seconds, while GPT-5.2 spends 209 seconds. These findings demonstrate that NSDS delivers approximately a 70 $\times$ acceleration in reasoning speed while maintaining superior accuracy. Specifically, for NSDS, the execution time of each processing stage is as follows: approximately 125 ms for YOLO-based object detection, 3.42 s for SQL generation, 115 ms for database execution, and less than 10 ms for result aggregation and output. In comparison, the GPT-5.2 model’s reasoning pipeline involves image understanding, semantic relationship extraction, and internal logical generation within the black-box language model. While this monolithic approach is functionally capable of completing the task, it suffers from prohibitive latency. Furthermore, its reliance on pure autoregressive reasoning fundamentally restricts its logical expressiveness: an LLM, in essence, is a next-token predictor that can only represent first-order predicate logic with a limited number of variables. Consequently, it struggles with multi-variable JOINS, recursive reasoning, and graph inference—operations that require the kind of systematic relational algebra that NSDS can handle.

B. Video Querying Task

In terms of the video querying task, we conducted the experiment on a dataset of 411 videos based on UCF101¹, each accompanied by descriptive labels. The task is to retrieve, from this large-scale video corpus, all videos that satisfy the following conditions: The video content is related to recreational activities and includes at least one frame where a bird appears to the right of a person, with a total duration less than 6 seconds. The specific steps of NSDS include: automatic SQL generation by an LLM based on the query conditions; SQL execution over an RDBMS for video metadata filtering; frame visual inspection using YOLO on the filtered video subset; and final result aggregation. As a baseline, we use GPT-5.2 alone to perform end-to-end video filtering. The comparison in terms of running time is shown in Table II. From the table, we observe that NSDS takes only 78.9 seconds, while GPT-5.2 requires 7047.80 seconds. These results demonstrate that NSDS achieves approximately an 90 $\times$ speedup in analytical query processing while maintaining superior accuracy. Specifically, the execution time breakdown for NSDS is as follows: 22.93 s for LLM-based SQL generation, 10.84 s for video metadata filtering within RDBMS, and 45.13 s for YOLO-based video analysis. In contrast, the GPT-5.2 baseline suffers from two primary bottlenecks: first, it lacks an efficient mechanism to rapidly filter out irrelevant videos; second, as

a massive multimodal foundation model, its visual analysis of video content is much slower and more compute-intensive than specialized, lightweight neural networks like YOLO.

It should be noted that these experiments only serve as a preliminary evaluation of NSDS, demonstrating its advantages in efficient and automatic neural-symbolic task decomposition and execution. Nevertheless, they confirm that NSDS offers a promising and practical pathway toward next-generation intelligent data management frameworks for mixed neural-symbolic workloads. With the challenges mentioned above properly addressed, we believe NSDS is well-positioned to tackle even more complex neural-symbolic applications.

VI. CONCLUSION AND FUTURE WORK

The prevailing paradigm of scaling Large Language and Multimodal Models is encountering an unsustainable bottleneck. For the next generation of AI, we believe a new paradigm of data systems is required that efficiently fuses the perceptual capabilities of neural networks with the verifiable logic of symbolic systems to better manage and analyze data. Given that real applications typically involve a sequence of interacting sub-tasks among different multimodal data systems (e.g., graph databases, relational databases, vector databases, image databases, etc.) across heterogeneous programming environments, we argue that existing data systems are inadequate for the dynamic, recursive, and iterative reasoning workflows essential for these complex scenarios. We thus introduce NSDS, a next-generation Neural-Symbolic Data System. In particular, its *hybrid framework combining both agentic and deterministic execution* provides a natural and effective paradigm. This architecture is uniquely suited to autonomously manage complex, multi-language task execution, error recovery, and continuous refinement across different data systems in heterogeneous environments, while still providing explainability and correctness guarantees. We have analyzed the core technical challenges in designing such a data system, outlining a comprehensive research roadmap that spans task decomposition, heterogeneous execution, multi-agent coordination, unified storage, continuous learning, and lineage auditing. Our preliminary evaluation, which demonstrated that NSDS can achieve approximately a 70 $\times$ acceleration in reasoning speed while maintaining superior accuracy over a monolithic LLM, validates the immense potential of this neural-symbolic approach. By tightly integrating neural perception with symbolic reasoning, our future work will focus on building and evolving NSDS as the foundational system supporting AI for Science and intelligence analysis.

VII. ACKNOWLEDGMENT

Ye Yuan is supported by the National Science Foundation of China (Grant Nos. 62225203, 62532001), the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (Grant No. JYB2025XDXM108), and the Beijing Natural Science Foundation (Grant No. L241010). Bo Tang is supported by the National Science Foundation of China (Grant No. 62422206). Zhaojing Luo

¹<https://www.crev.ucf.edu/data/UCF101.php>

is supported by the National Key Research and Development Program of China (2025YFE0216400), the National Natural Science Foundation of China (62472030). Boyang Li is supported by the National Science Foundation of China (Grant No. 62394332, 62572050) and the Liaoning Revitalization Talents Program (Grant No. XLYC2204005).

REFERENCES

- [1] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *NeurIPS*, vol. 35, 2022, pp. 27 730–27 744.
- [2] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "GPT-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.
- [3] B. C. Ooi, S. Cai, G. Chen, Y. Shen, K.-L. Tan, Y. Wu, X. Xiao, N. Xing, C. Yue, L. Zeng *et al.*, "Neurdb: an ai-powered autonomous data system," *Science China Information Sciences*, vol. 67, no. 10, p. 200901, 2024.
- [4] Z. Zhao, S. Cai, H. Gao, H. Pan, S. Xiang, N. Xing, G. Chen, B. C. Ooi, Y. Shen, Y. Wu, and M. Zhang, "Neurdb: On the design and implementation of an ai-powered autonomous database," *CIDR*, 2025.
- [5] "EvaDB: Database system for AI-powered apps," 2025. [Online]. Available: <https://github.com/georgia-tech-db/evadb>
- [6] F. Del Buono, M. Paganelli, P. Sottovia, M. Interlandi, and F. Guerra, "Transforming ML predictive pipelines into SQL with MASQ," in *SIGMOD*, 2021, pp. 2696–2700.
- [7] K. Park, K. Saur, D. Banda, R. Sen, M. Interlandi, and K. Karanasos, "End-to-end optimization of machine learning prediction queries," in *SIGMOD*, 2022, pp. 587–601.
- [8] B. Yuan, D. Jankov, J. Zou, Y. Tang, D. Bourgeois, and C. Jermaine, "Tensor relational algebra for distributed machine learning system design," *PVLDB*, vol. 14, no. 8, 2021.
- [9] L. Zhou, Q. Lin, K. Chowdhury, S. Masood, A. E. Eichenberger, H. Min, A. Sim, J. Wang, Y. Wang, K. Wu, B. Yuan, and J. Zou, "Serving Deep Learning models from relational databases," *EDBT*, vol. 27, no. 3, pp. 717–724, 2024.
- [10] S. Luo, Z. J. Gao, M. Gubanov, L. L. Perez, and C. Jermaine, "Scalable linear algebra on a relational database system," *SIGMOD Record*, vol. 47, no. 1, pp. 24–31, 2018.
- [11] Y. Lou, C. Lei, X. Qin, Z. Wang, C. Faloutsos, R. Anubhai, and H. Rangwala, "Datalore: Can a large language model find all lost scrolls in a data repository?" in *ICDE*, 2024, pp. 5170–5176.
- [12] Y. Yuan, B. Tang, T. Zhou, Z. Zhang, and J. Qin, "nsdb: Architecting the next generation database by integrating neural and symbolic systems (vision)," *PVLDB*, vol. 17, no. 11, pp. 3283–3289, 2024.
- [13] R. Liu, H. Jiang, X. Yan, B. Tang, and J. Li, "Polyg: Adaptive graph traversal for diverse graphrag questions," 2025. [Online]. Available: <https://arxiv.org/abs/2504.02112>
- [14] "AI Functions on Databricks," 2025. [Online]. Available: <https://docs.databricks.com/aws/en/large-language-models/ai-functions>
- [15] "Amazon Redshift ML is Now Generally Available – Use SQL to Create Machine Learning Models and Make Predictions From Your Data," 2021. [Online]. Available: <https://aws.amazon.com/blogs/aws/amazon-redshift-ml-is-now-generally-available-use-sql-to-create-machine-learning-models-and-make-predictions-from-your-data/>
- [16] "Sql-only llm for text generation using vertex ai model in bigquery," 2023. [Online]. Available: <https://cloud.google.com/blog/products/ai-machine-learning/llm-with-vertex-ai-only-using-sql-queries-in-bigquery>
- [17] N. Armatatzoglou, S. Basu, N. Bhanoori, M. Cai, N. Chainani, K. Chinta, V. Govindaraju, T. J. Green, M. Gupta, S. Hillig, E. Hotinger, Y. Leshinksky, J. Liang, M. McCredy, F. Nagel, I. Pandis, P. Parchas, R. Pathak, O. Polychroniou, F. Rahman, G. Saxena, G. Soundararajan, S. Subramanian, and D. Terry, "Amazon redshift re-invented," in *SIGMOD*, 2022, p. 2205–2217.
- [18] R. Salazar-Díaz, B. Glavic, and T. Rabl, "Inferdb: In-database machine learning inference using indexes," *PVLDB*, vol. 17, no. 8, p. 1830–1842, 2024.
- [19] "Postgresml," 2025. [Online]. Available: <https://postgresml.org/>
- [20] Z. Luo, S. H. Yeung, M. Zhang, K. Zheng, L. Zhu, G. Chen, F. Fan, Q. Lin, K. Y. Ngiam, and B. C. Ooi, "Mlcask: Efficient management of component evolution in collaborative data analytics pipelines," in *ICDE*, 2021, pp. 1655–1666.
- [21] X. Bi, D. Chen, G. Chen, S. Chen, D. Dai, C. Deng, H. Ding, K. Dong, Q. Du, Z. Fu *et al.*, "Deepseek llm: Scaling open-source language models with longtermism," *arXiv preprint arXiv:2401.02954*, 2024.
- [22] Y. Fan, Z. He, T. Ren, C. Huang, Y. Jing, K. Zhang, and X. S. Wang, "Metasql: A generate-then-rank framework for natural language to sql translation," in *ICDE*, 2024, pp. 1765–1778.
- [23] P. Papotti, "Large language models as storage for sql querying," in *ICDE*, 2024, pp. 5657–5658.
- [24] Y. Deng, Y. Wang, L. Cao, L. Qiao, Y. Wang, J. Xu, Y. Yan, and S. Madden, "Outlier summarization via human interpretable rules," *PVLDB*, vol. 17, no. 7, pp. 1591–1604, 2024.
- [25] Z. Luo, S. Cai, J. Gao, M. Zhang, K. Y. Ngiam, G. Chen, and W.-C. Lee, "Adaptive lightweight regularization tool for complex analytics," in *ICDE*, 2018, pp. 485–496.
- [26] Z. Luo, S. Cai, C. Cui, B. C. Ooi, and Y. Yang, "Adaptive knowledge driven regularization for deep neural networks," in *AAAI*, vol. 35, no. 10, 2021, pp. 8810–8818.
- [27] J. Li and R. Cotterell, "Characterizing the expressivity of fixed-precision transformer language models," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [28] P. Barcelo, A. Kozachinskiy, A. W. Lin, and V. Podolskii, "Logical languages accepted by transformer encoders with hard attention," in *The Twelfth International Conference on Learning Representations*, 2024.
- [29] W. Merrill and A. Sabharwal, "The expressive power of transformers with chain of thought," in *The Twelfth International Conference on Learning Representations*, 2024.
- [30] M. Zhang, Z. Ji, Z. Luo, Y. Wu, and C. Chai, "Applications and challenges for large language models: From data management perspective," in *ICDE*, 2024, pp. 5530–5541.
- [31] G. Li, "Data agent: A holistic architecture for orchestrating data+ai ecosystems," in *ICDE*, Hong Kong, China, 2025, keynote Talk.
- [32] Z. Tang, W. Wang, Z. Zhou, Y. Jiao, B. Xu, B. Niu, D. Zhou, X. Zhou, G. Li, Y. He, W. Zhou, Y. Song, C. Tan, X. Yang, C. Liu, B. Wang, C. He, X. Wang, and F. Wu, "Llm/agent-as-data-analyst: A survey," 2025.
- [33] K. Team, Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding *et al.*, "Kimi k2: Open agentic intelligence," 2026.
- [34] Z. Shi, S. Gao, L. Yan, Y. Feng, X. Chen, Z. Chen, D. Yin, S. Verberne, and Z. Ren, "Tool learning in the wild: Empowering language models as automatic tool agents," in *Proceedings of the ACM on Web Conference*, 2025, p. 2222–2237.
- [35] C.-Y. Chang, Z. Jiang, V. Rakesh, M. Pan, C.-C. M. Yeh, G. Wang, M. Hu, Z. Xu, Y. Zheng, M. Das, and N. Zou, "Main-rag: Multi-agent filtering retrieval-augmented generation," 2024.
- [36] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu, C. Zhang, D. Li, J. Chen, J. Zhang, J. Wang, L. Zhang, L. Zhang, M. Yang, M. Zhuge, T. Guo, T. Zhou, W. Tao, R. Tang, X. Lu, X. Zheng, X. Liang, Y. Fei, Y. Cheng, Y. Ni, Z. Gou, Z. Xu, Y. Luo, and C. Wu, "Data interpreter: An LLM agent for data science," in *Findings of the Association for Computational Linguistics: ACL 2025*, Jul. 2025, pp. 19 796–19 821.
- [37] W. Wang, Y. Yang, and F. Wu, "Towards data-and knowledge-driven ai: a survey on neuro-symbolic computing," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [38] D. Cheng, S. Xiang, C. Shang, Y. Zhang, F. Yang, and L. Zhang, "Spatio-temporal attention-based neural network for credit card fraud detection," in *AAAI*, vol. 34, no. 1, 2020, pp. 362–369.
- [39] D. Kang, F. Romero, P. D. Bailis, C. Kozyrakis, and M. Zaharia, "Viva: An end-to-end system for interactive video analytics," in *CIDR*, 2022.
- [40] M. Urban and C. Binnig, "Eleet: Efficient learned query execution over text and tables," *PVLDB*, vol. 17, no. 13, pp. 4867–4880, 2024.
- [41] Z. Luo, S. Cai, Y. Wang, and B. C. Ooi, "Regularized pairwise relationship based analytics for structured data," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, pp. 1–27, 2023.
- [42] L. Wang, M. Zhang, and Z. Luo, "Aixelnet: A pre-trained model with table-aware adaptation for structured data prediction," *Proceedings of the ACM on Management of Data*, vol. 3, no. 6, 2025.
- [43] A. Khan, "Knowledge graphs querying," *SIGMOD Record*, vol. 52, no. 2, p. 18–29, 2023.

- [44] X. Zhu, Z. Li, X. Wang, X. Jiang, P. Sun, X. Wang, Y. Xiao, and N. J. Yuan, "Multi-modal knowledge graph construction and application: A survey," *TKDE*, vol. 36, no. 2, pp. 715–735, 2024.
- [45] M. Jungmair, A. Kohn, and J. Giceva, "Designing an open framework for query optimization and compilation," *PVLDB*, vol. 15, no. 11, pp. 2389–2401, 2022.
- [46] P. M. Grulich, B. Sebastian, S. Zeuch, J. Traub, J. v. Bleichert, Z. Chen, T. Rabl, and V. Markl, "Grizzly: Efficient stream processing through adaptive query compilation," in *SIGMOD*, 2020, pp. 2487–2503.
- [47] G. Fan, C. Zhang, J. Chen, B. Li, Z. Xu, Y. Li, L. Peng, and Z. Gong, "Field-aware variational autoencoders for billion-scale user representation learning," in *ICDE*, 2022, pp. 3413–3425.
- [48] Y. Zhang, L. Chen, S. Yang, M. Yuan, H. Yi, J. Zhang, J. Wang, J. Dong, Y. Xu, Y. Song, Y. Li, D. Zhang, W. Lin, L. Qu, and B. Zheng, "Picasso: Unleashing the potential of gpu-centric training for wide-and-deep recommender systems," in *ICDE*, 2022, pp. 3453–3466.
- [49] X. Zhou, G. Li, Z. Sun, Z. Liu, W. Chen, J. Wu, J. Liu, R. Feng, and G. Zeng, "D-Bot: Database diagnosis system using large language models," *PVLDB*, vol. 17, no. 10, pp. 2514–2527, 2024.
- [50] Y. Zhang, Y. Chronis, J. M. Patel, and T. Rekatsinas, "Simple adaptive query processing vs. learned query optimizers: Observations and analysis," *PVLDB*, vol. 16, no. 11, pp. 2962–2975, 2023.
- [51] M. Z. Pan, M. Cemri, L. A. Agrawal, S. Yang, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, K. Ramchandran, D. Klein, J. E. Gonzalez, M. Zaharia, and I. Stoica, "Why do multiagent systems fail?" in *ICLR*, 2025.
- [52] M. Ma, Z. Yin, S. Zhang, S. Wang, C. Zheng, X. Jiang, H. Hu, C. Luo, Y. Li, N. Qiu, F. Li, C. Chen, and D. Pei, "Diagnosing root causes of intermittent slow queries in cloud databases," *PVLDB*, vol. 13, no. 8, pp. 1176–1189, 2020.